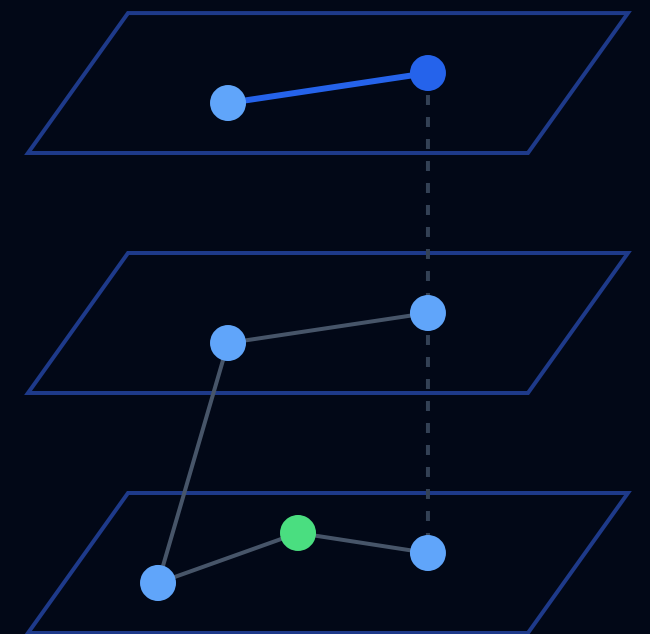


ENGINEERING EXPLAINER

How HNSW Works

Hierarchical Navigable Small Worlds — the graph index behind fast approximate nearest-neighbor search.



The Nearest-Neighbor Problem

Given a query embedding, find the most similar vectors in a collection. The exact answer requires a distance computation against **every single vector**.

Brute force, per query

$O(N \cdot d)$

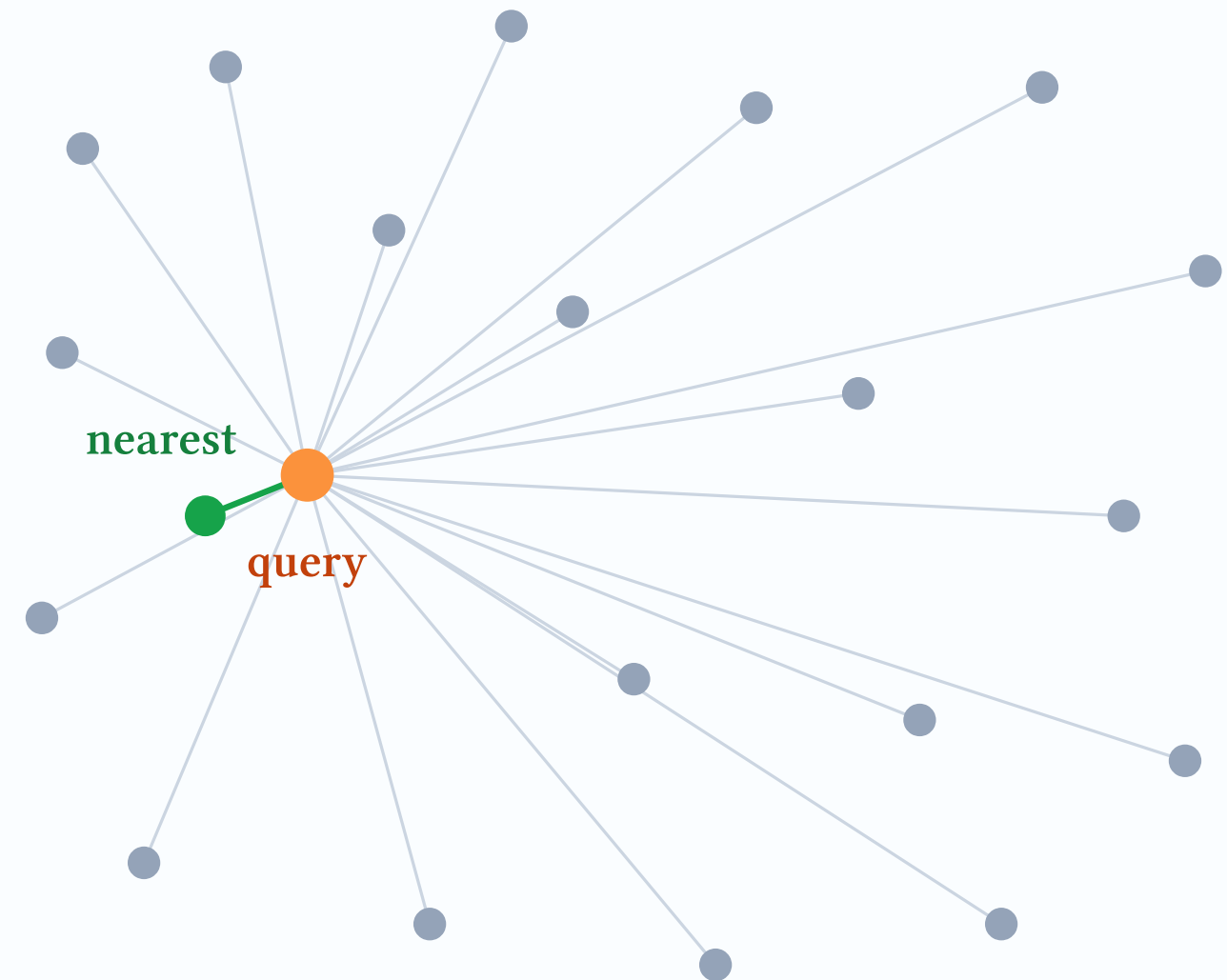
100M vectors × 1,536 dims
— seconds

HNSW, per query

$\sim O(\log N)$

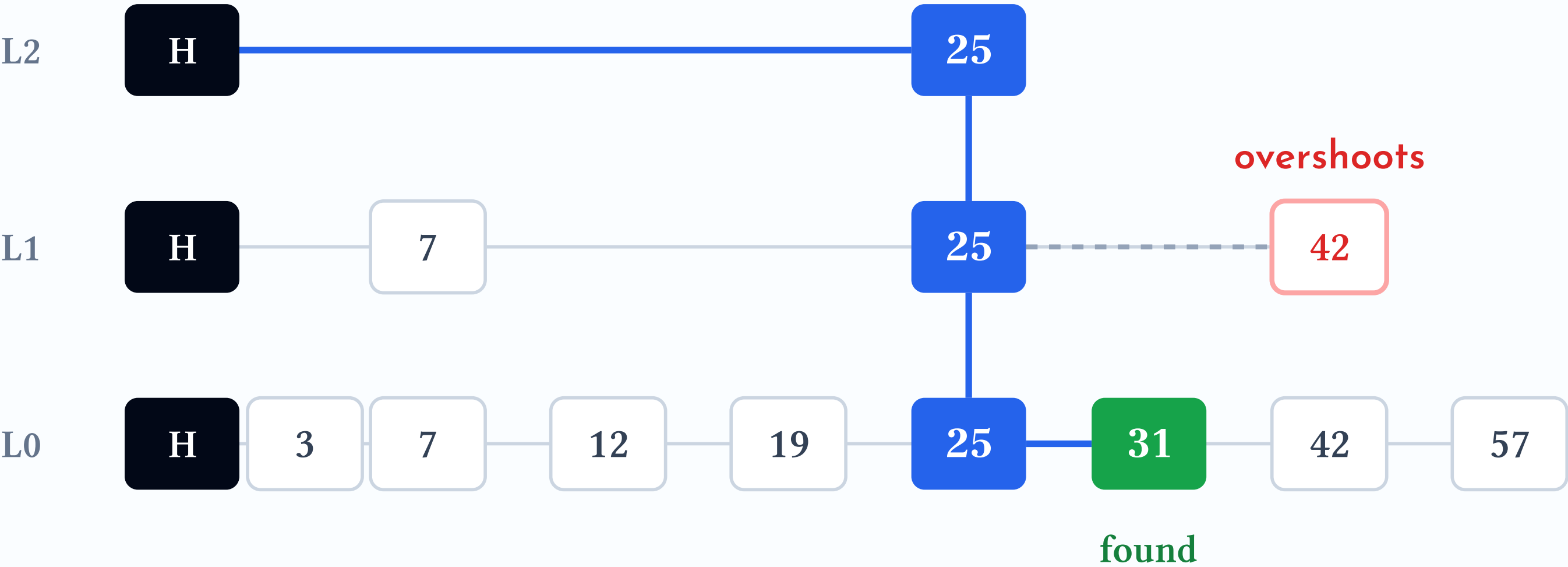
Milliseconds, at 95–99%
recall

The catch: it is *approximate* — it occasionally misses the true nearest neighbor. For search, RAG, and recommendations, that trade is almost always worth it.



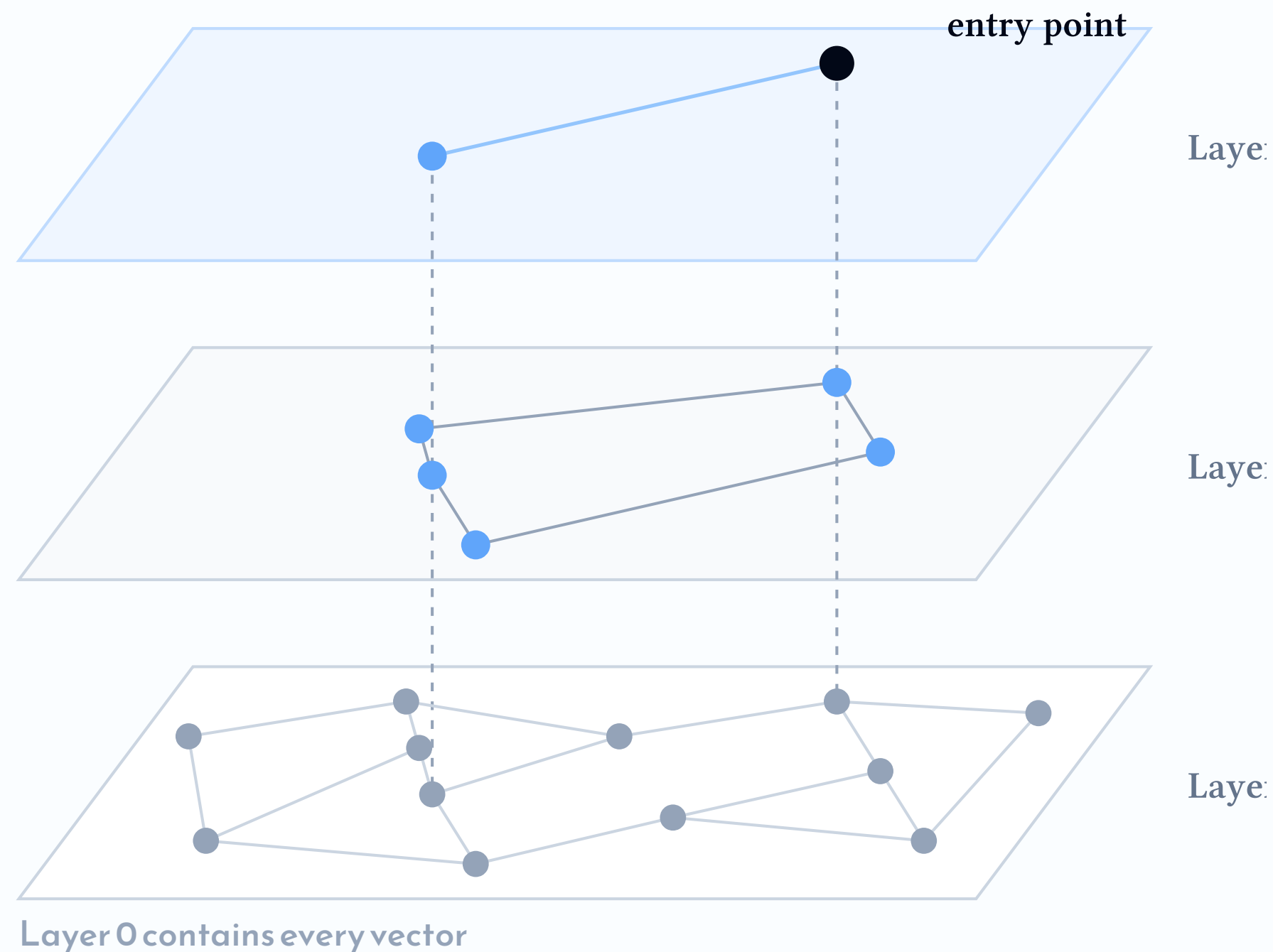
Skip Lists

Sparse “express lanes” sit above the full list. Start on the top lane and take big jumps; drop down a lane whenever the next jump would overshoot. Finding **31** takes 3 hops instead of scanning 7 nodes.



HNSW applies the same “coarse first, fine last” idea to a proximity **graph** in high-dimensional space.

The Layered Graph



Every vector lives in layer 0

The bottom layer is a proximity graph over the full dataset — each node linked to its nearest neighbors.

Higher layers are random samples

Each node is promoted upward with exponentially decaying probability, so every layer up is ~an order of magnitude sparser.

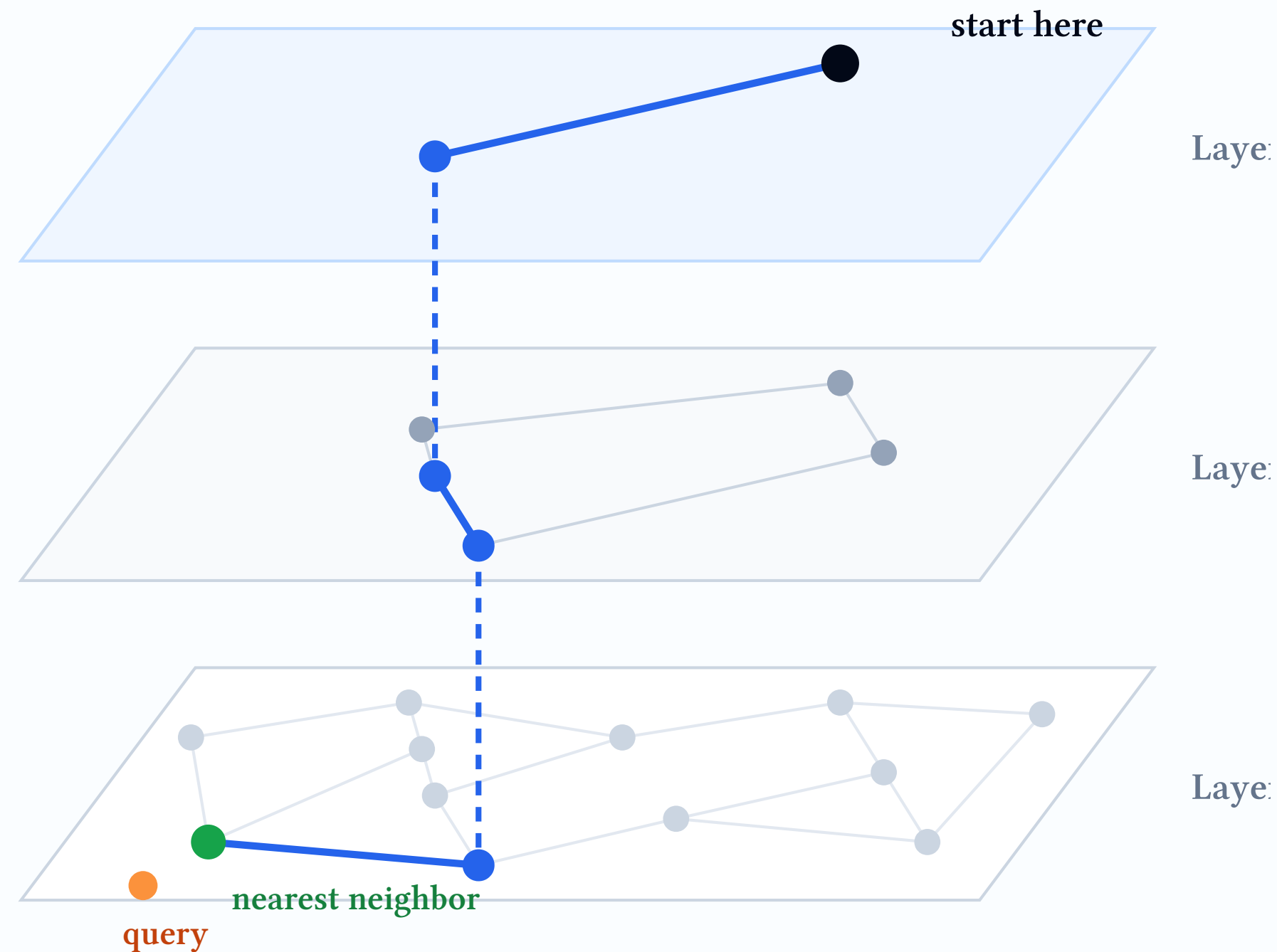
Sparse layers = long-range links

Nearest neighbors in a sparse layer are far apart in space — those edges are the express lanes.

One entry point on top

The highest-layer node is where every search begins.

Searching the Index



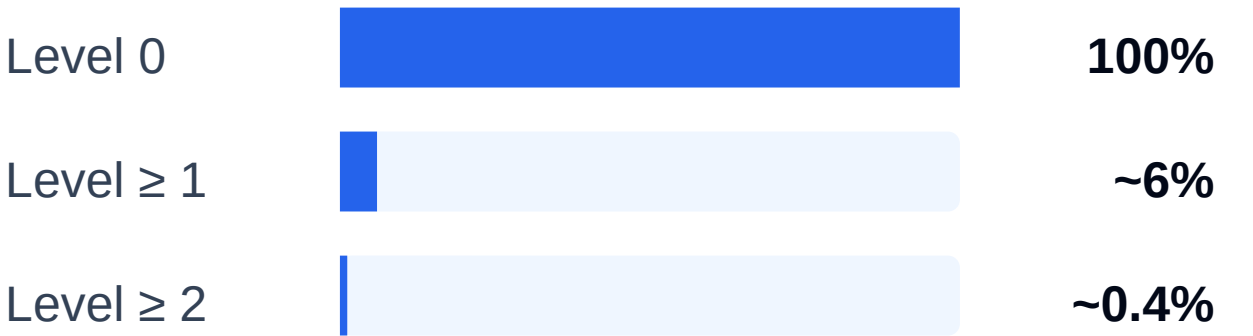
- 1 Start at the entry point on the top layer.
- 2 Greedily hop to whichever neighbor is closest to the query.
- 3 When no neighbor is closer, drop down one layer and repeat.
- 4 On layer 0, widen to a small candidate beam (efSearch) and return the best k.

Coarse layers cover distance; layer 0 does the fine positioning. Only ~dozens of nodes are visited out of millions.

Inserting a Vector

1 · Draw a random top level

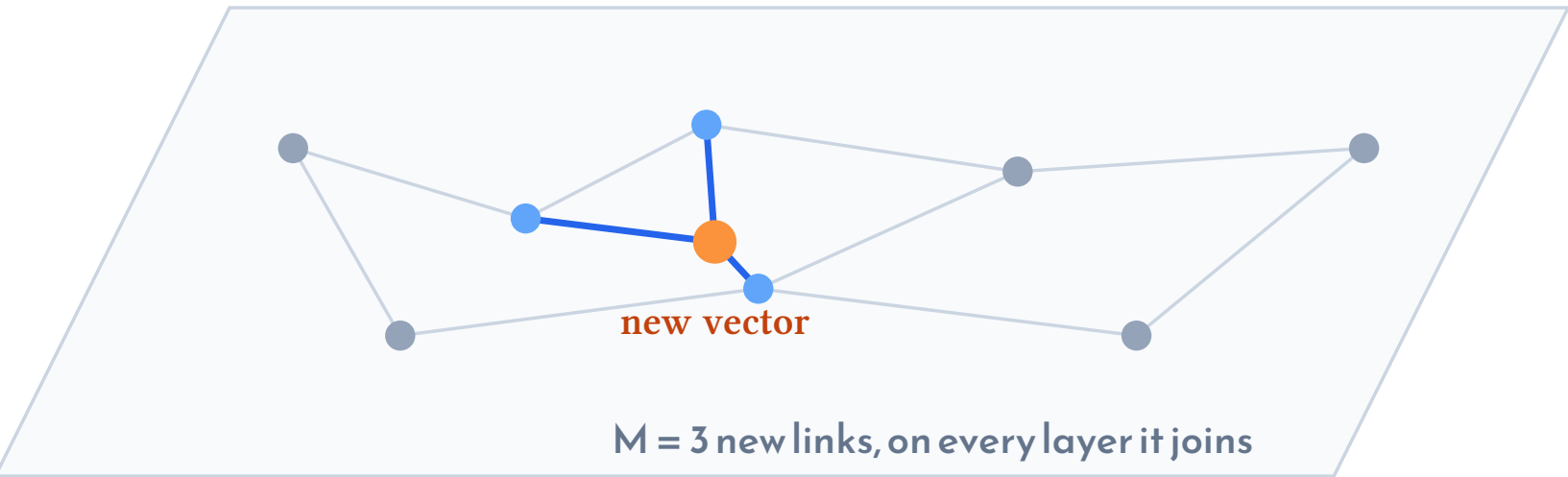
Exponential distribution — the hierarchy emerges from a coin flip, no balancing needed.



Probabilities shown for M = 16

2 · Search down, then link to the M nearest

A normal search finds the neighborhood; the new node connects to its M closest nodes on each layer it belongs to.



If a neighbor exceeds its link budget, its worst edge is pruned — degree stays bounded.

The Three Parameters

M

Links per node

Graph connectivity. Higher M → better recall and more robust graphs, but memory and build time grow linearly with it.

Typical: **12 – 48** · fixed at build

efConstruction

Build-time beam width

How many candidates are examined when choosing each new node's neighbors. Higher → better graph quality, slower indexing.

Typical: **100 – 400** · fixed at build

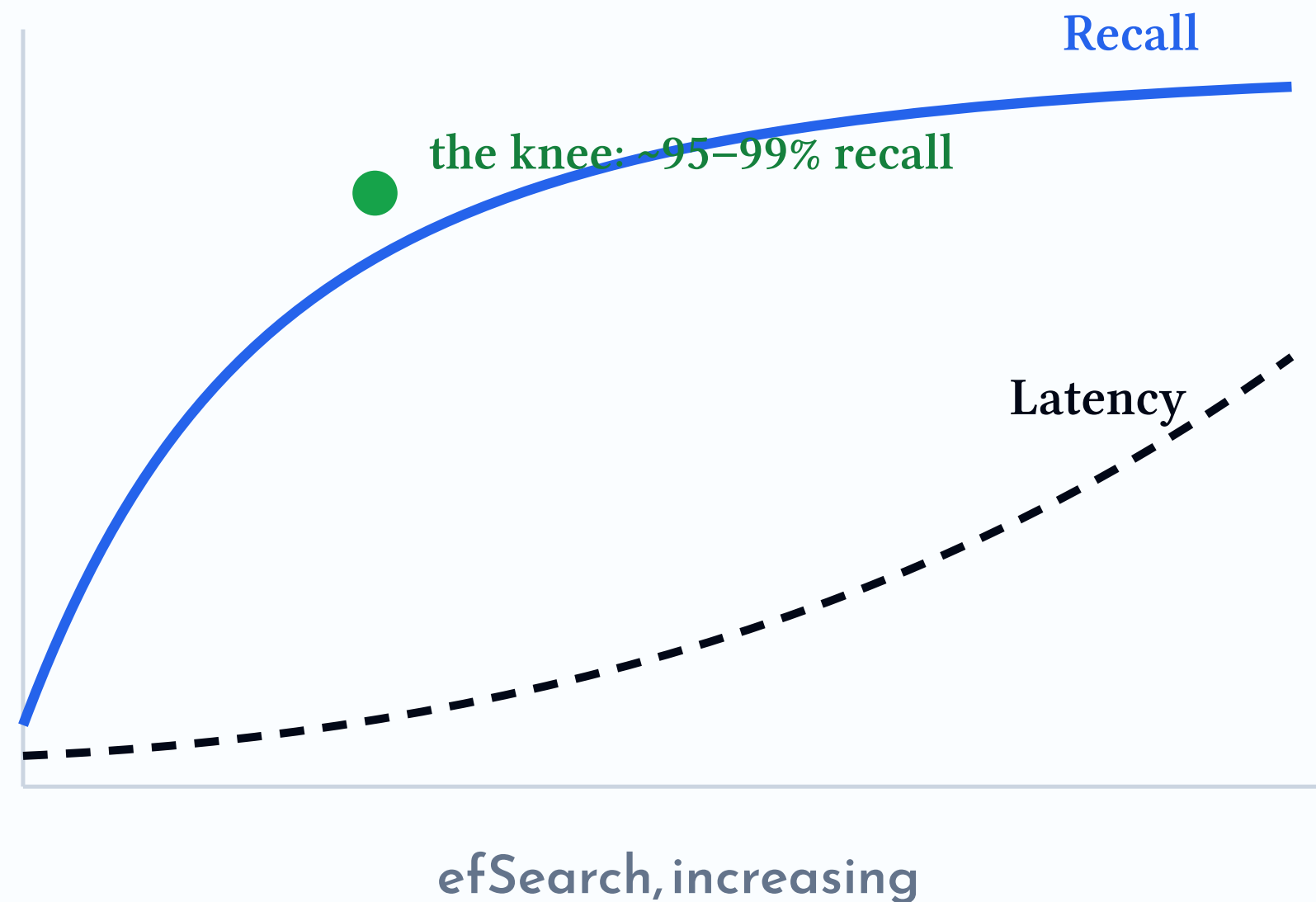
efSearch

Query-time beam width

How many candidates the layer-0 search keeps alive. The knob you turn in production — raise it for recall, lower it for latency.

Typical: **k – 512** · tunable per query

Recall vs. Speed



What to remember

- 01** HNSW is a skip list generalized to a proximity graph: coarse layers travel far, layer 0 refines.
- 02** Search is greedy descent from a single entry point — $\sim O(\log N)$ hops, no training step.
- 03** The hierarchy comes from a random level draw at insert time; M links bound every node's degree.
- 04** In production, efSearch is your dial: sweep it, plot this curve, pick your